

Advanced-programming report

The first part of the code on which I really stumbled upon, was in the function `get_all_neighbours()`. Indeed, at first I used the function `find_neighbourhood` with the positions of the predators to calculate their coherence. This method mainly worked, but predators tended to try and stick together too much, even more when no boids were near enough and then they would just stop to hunt and slowly move together aimlessly. It took me some time to understand that by using the already existing function, the predators would consider the boids but also other predators to compute their coherence, which is why I chose this solution to rebuild the `KDTree` and redo the `query_ball_point` by using the positions of the boids only for the formers, and the positions of the predators only for the latter.

```
318         # Check if there are predators
319         if len(self.predator_list) > 0:
320
321             # get positions of predators
322             positions_p = np.array([p.position for p in self.predator_list])
323
324             # positions of boids around predators
325             tree_boids = KDTree(positions_b)
326             neighbs_pred = tree_boids.query_ball_point(positions_p, r=self.r_pred)
327
328             self.neighb_positions_pred = {i: positions_b[n] for i, n in enumerate(neighbs_pred)}
329
330             # positions of predators around predators for repulsion
331             neighbs_p = self.find_neighbourhood(positions_p, r=self.r_repulsion*3)
332             self.neighb_positions_pred_pred = {i: positions_p[n] for i, n in enumerate(neighbs_p)}
333
334             # positions of predators around boids for repulsion
335             tree_pred = KDTree(positions_p)
336             neighbs_pred_rep = tree_pred.query_ball_point(positions_b, r=self.r_repulsion*1.5)
337
338             self.neighb_positions_repulsion_pred = {i: positions_p[n] for i, n in enumerate(neighbs_pred_rep)}
```

In this block :

Line 319 : We first check if there are actually predators in the simulation otherwise it would crash.

Line 322 : We get the positions of all the predators in an array.

Line 325 : Construction of a k-d tree with the positions of the boids (`positions_b` is obtained the same way as `positions_p` but with the boids).

Line 326 : Computation of all the boids that are present inside a radius `r_pred` of the predators.

Line 328 : Create a dictionary that associates to each predator by its index, its neighbouring boids.

Line 331 : Using the function `find_neighbourhood`, we construct a k-d tree with the positions of the predators, and then compute the predators that are present inside a

radius `r_repulsion` (times 3 so that predators repulse further away from one another, so it helps them not sticking too much between each other and go hunt on their own).

Line 332 : Create a dictionary that associates to each predator by its index, its neighbouring predators.

Line 335 : Construction of a k-d tree with the positions of the predators.

Line 336 : Computation of all the predators that are present inside a radius `r_repulsion` (times 1.5 so they try to avoid predators further away than other boids) of the predators.

Line 338 : Create a dictionary that associates to each boid by its index, its neighbouring predators.

After a better understanding of the function and `KDTree`, I honestly can't even remember what code I used for the predators to detect boids as well as other predators. I used the functions in a very naive way when I first tried to play with the computation of the neighbours.

Seeing this block, I can see now that I could have just modified the `find_neighbourhood` function to use two different inputs for the positions in `KDTree` and `query_ball_point`, so the code could take a little less space. Saying this I now want to modify it but I'm running out of time and still have to finish this report. I may do it later and push it on github after due time.

Another problem that occurred after coding this, is that when a predator spawned far away from any boid, would have some trouble computing their neighbours as they all would be outside of any radius for coherence or separation. After a long time of printing positions and velocities for predators, it seemed like the function would indicate velocity of `[NaN, NaN]` when no neighbours were found, and the predator would be teleported out of the screen in position `[0, 0]` (still don't understand why) and would be stuck forever without any speed to move. I then simply added a check if there were any boids or predators in sight of each predator, and if that was not the case the coherence and separation would be both `[0, 0]` so the new velocity simply adds nothing to the current velocity and doesn't cause this bug anymore. In result, when a predator spawn far away from any boid or predator, it simply moves very slowly until another unit enters its radius for coherence or repulsion.

```
380     self.get_all_neighbours()
381     new_velocity = []
382     for b_index, p in enumerate(self.predator_list):
383         # compute coherence only if there are boids in sight. Added to avoid NaN errors and predator staying stuck out of the screen with NaN velocity.
384         if len(self.neighb_positions_pred[b_index]) > 0:
385             C = p.coherence(b_index, self.neighb_positions_pred[b_index], p.position, self.c_c*2)
386         else:
387             C = np.zeros(2)
388         # compute separation from other predators so that they don't stack and stay less in the same place
389         if len(self.neighb_positions_pred_pred[b_index]) > 0:
390             S = p.separation(b_index, self.neighb_positions_pred_pred[b_index], p.position, self.c_s)
391         else:
392             S = np.zeros(2)
393         new_velocity.append(p.velocity + self.dt * (C + S))
```

Line 380 : Call of the `get_all_neighbours` function that we partially saw earlier to get all the different types of neighbours.

Line 381 : Create an empty list that will later contain the `new_velocity`

Line 382 : Create a loop that goes through every predator

Line 384 : Condition, check if this predator has any neighbouring boids

Line 385 : If it has at least one, then it computes its coherence using the function defined in `_predator.py`.

Line 386/387 : If the predator has no neighbouring boids, then the coherence is `[0, 0]`

Line 389 : Condition, check if this predator has any neighbouring predators

Line 390 : If it has at least one, then it computes its separation using the function defined in `_predator.py`.

Line 391/392 : If the predator has no neighbouring predators, then the separation is `[0, 0]`.

Line 393 : We add at the end of the `new_velocity` list the value of the current velocity of the predator, plus the coherence and separation times the time interval `dt`.

Another part of the code that I am pretty happy with, is the management of boids getting eaten. Basically in the pygame loop, when a predator is within a certain reach of a boid, said boid is deleted from the list. It could have ended here but I wanted more, so I added two parameters : “eaten” and “cooldown” to the class `Predator`, which tracks if a predator has eaten recently and the time elapsed since it did, and “score” to the class `Flock` that tracks the number of boids eaten.

```

152         # Delete boids that are in the same position as predators
153         for p in self.predator_list:
154             if np.linalg.norm(current_pos - p.position) < 10 and not p.eaten:
155                 self.boid_list.remove(b)
156                 p.eaten = True
157                 self.score += 1 # increment score when a boid is eaten
158                 eat_text = eat_image[np.random.randint(0, len(eat_image))] # add eat image when a boid is eaten
159                 self.eat_image.append((eat_text, current_pos.copy(), pygame.time.get_ticks(), 1500)) # append image with its position, start time and duration
160                 # If there is no bird left, game is paused
161                 if len(self.boid_list) == 0:
162                     game_paused = True
163                     end_time = pygame.time.get_ticks()
164                 break

```

This is in the part of the pygame loop responsible for the movement of the boids.

Line 153 : Goes through every predator.

Line 154 : If the difference of the position of the predator and the boid being moved is small enough, so if they are close enough, and the predator has not eaten too recently

Line 155 : Then the boid is removed from the list.

Line 156 : The state “eaten” of the predator is set to `True` so it will not be able to eat again yet.

Line 157 : The score is incremented of one.

Line 158/159 : A random image is added to the `Flock` parameter “`eat_image`”, which originally is an empty list. This image is displayed at the position of the boid that has just been eaten. Another part of the code manage so that this image fade out and is deleted of the list `eat_image` in 1.5s

Line 161 : Another check, is after deleting the boid, there is no boid left, then the simulation is paused and the time at which the simulation ends is stored.

Most of this block is only for aesthetic purposes, with the displaying of the image, and the managing of the end of simulation, but it was really interesting to use the classes more by myself, by introducing new parameters to Predator and Flock. It allowed me to do things I wouldn't have known how to do otherwise, and in the end it was really satisfying to play with the different parameters as states of each class to manage each of them at the same time.

```
122 # manage cooldown after eating (predator will be slower for a while, cf one_step_move_predator)
123 if not game_paused:
124     if p.eaten:
125         p.cooldown += 1
126         if p.cooldown >= 180: # cooldown of 3 seconds at 60 fps
127             p.eaten = False
128             p.cooldown = 0
```

This is in the part of the pygame loop responsible for the movement of the predators.

Line 123 : Condition if the simulation is paused, so the cooldown of the predators does not reduce while it should not

Line 124 : Condition if the predator moved currently has eaten

Line 125 : Then increment its cooldown value to 1

Line 126 : Condition if the cooldown is superior or equal to 180 frames (or 3s)

Line 127 : Then the condition eaten of this predator is set to False so it can eat again

Line 218 : And its cooldown is reseted to 0 for the next time it will eat a boid

Having the parameters eaten and cooldown as parameters of the class Predators allow us to manage the state of each predator independently inside the loop where we move them one by one. When I first thought about adding a cooldown to the predators, I was scared at first of the presumed complexity of the task when only thinking about it without what all the classes allow. I was then happily surprised to quickly find a simple solution to this problem, that allowed me to spend more time on the visual polishing of the simulation, even if not necessary at all.

Adding the possibility to pause the simulation was also very interesting. It would be tricky to show here as it is managed in multiple separated parts of the code, but basically when an event is detected, and this event is the space bar being pressed, a True/False object game_paused is inverted so to switch between paused and unpaused at each press of the space bar. While stopping the flock from moving was easy, as each tick the state of game_paused is checked, and the function move_flock is called only when the simulation is unpaused, managing the fading of the images that are displayed when the simulation is paused was pretty complicated. Indeed, even if the simulation is considered paused for the movement of the boids and predators, the ticks in pygame continue to scroll. As the fading is calculated based on the ticks, at first pausing the simulation would not pause the fading, as it does not pause the pygame ticks or it would crash (I tried...). So I had to adjust all the timings and ticks counting so that the fading freezes during pause and resumes when unpausing. It almost works perfectly, but there seems to be a brutal loss of

opacity when the same image displayed undergoes multiple pauses. Most often, the first pause works as it should and unpausing works too, but triggering a second pause causes the opacity to suddenly drop, and a third same thing, so much that it completely disappears even if the pauses are triggered way quicker than the normal 1.5s of fade time.

After explaining all that, it may be now clear that I spent a lot of time on some visual details. While they were not the most useful or relevant things to what was expected, I also tried to have some fun and got a little carried away, really wanting things to look right when they did not. All in all, retrospectively, I could have spent more time on the simulation in itself. As of now, the simulation only has one type of bird, and one type of predator. It could have been interesting to add more, as it's really simple to just create new classes and tweak a few numbers here and there to make them have different kinds of behaviour. I'm thinking maybe of a type of bird that could "panic" when another bird is eaten in a certain radius, gaining increased velocity to fly away from the predator faster than the others. Or another type of inbetween bird and predator, that acts as a bird, but could target and go hunt the predators when they are "tired", in cooldown right after eating. This class would need to check for the presence of predators with its parameter "eaten" True in a certain radius in his `one_step_move` function, having two separate behaviours possible, when checking for its neighbours, if one is a predator in a "eaten = True" state, would switch to an aggressive behaviour.

It is not part of the project at all, but I would also be curious to try to implement a "player", a predator that could be controlled by the user, still with a cooldown after eating a bird. It would also need its own class, but from what I saw it would need far more pygame event management than what I did in this project, and I'm not sure how I could manage the acceleration. Maybe allow the buildup of speed if the player goes in a straight line or turn with less than a certain angle, but a sharp turn would greatly decrease its speed. The birds would need to recognize it as a predator, but the behaviour would be the same so it's just a matter of integrating it in the neighbourhood computation. The player being apart from other predators, it would need its own score counting, to see how well you are doing compared to autonomous behaviour. Which would just need to shift the score from the Flock class to the Predator and Player class to manage them independently.

More on the original simulation, it could be interesting to be able to extract the trajectory of the different birds and/or predators during the simulation. With another popup message box, you could have the option to extract a table with all the positions of all birds and predators throughout the simulation. With the tables it would then be easy to plot the trajectories afterward. I don't know if this would really slow

the process of the simulation, but I'm thinking of extracting the position of each unit every time the `move_flock()` function is called, and storing it in its personal table. Maybe offering to extract the tables as csv using some module, or immediately displaying trajectories after the end or when the simulation is paused.

All of these seem like nice ideas, but I have far too little time. Maybe you will see updates on github in the future, just for fun. Who knows ?

As a side note, I should say that I encountered some problems while trying to run the code on another computer than the main one I used. The pull was ok, creating a new environment as well, but when trying to "pip install -e .", the console always said an error message, as the python version was apparently not the right one (3.14 instead of 3.9.13). At first it was because the wright version was not installed on my computer, but even after it was installed, I could only run it by typing "py -m pip install -e ." instead. Even if the python version (py --version) was 3.9.13. I don't understand why it is not working properly on the other computer, while it's working perfectly fine on the first. As I wrote this, I tried one last time with another version (3.13.9, because I think pygame is ok with this version, just not the 3.14) and it did not work even with the other method that worked for the 3.9.13. And after trying to go back to 3.9.13 because at least it worked, it doesn't even work anymore. I checked the Path for the environment variable, and every version of python I want to use is on it. I must have missed something really dumb, the python version of every new environment before installing anything is 3.14. I think this computer is cursed.

I'm thinking about burning it.

The other is still fine though, even if I'm scared to install python 3.13.9 to try. I want to cry so I'll not try.